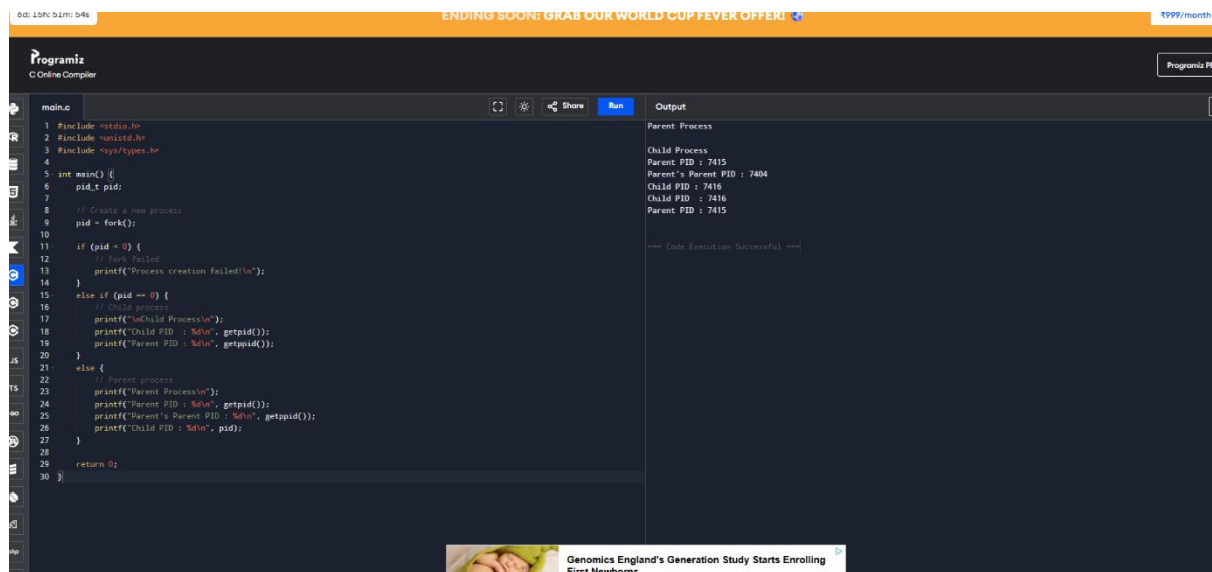


1. Create a new process by invoking the appropriate system call. Get the process identifier of the currently running process and its respective parent using system calls and display the same using a C program.



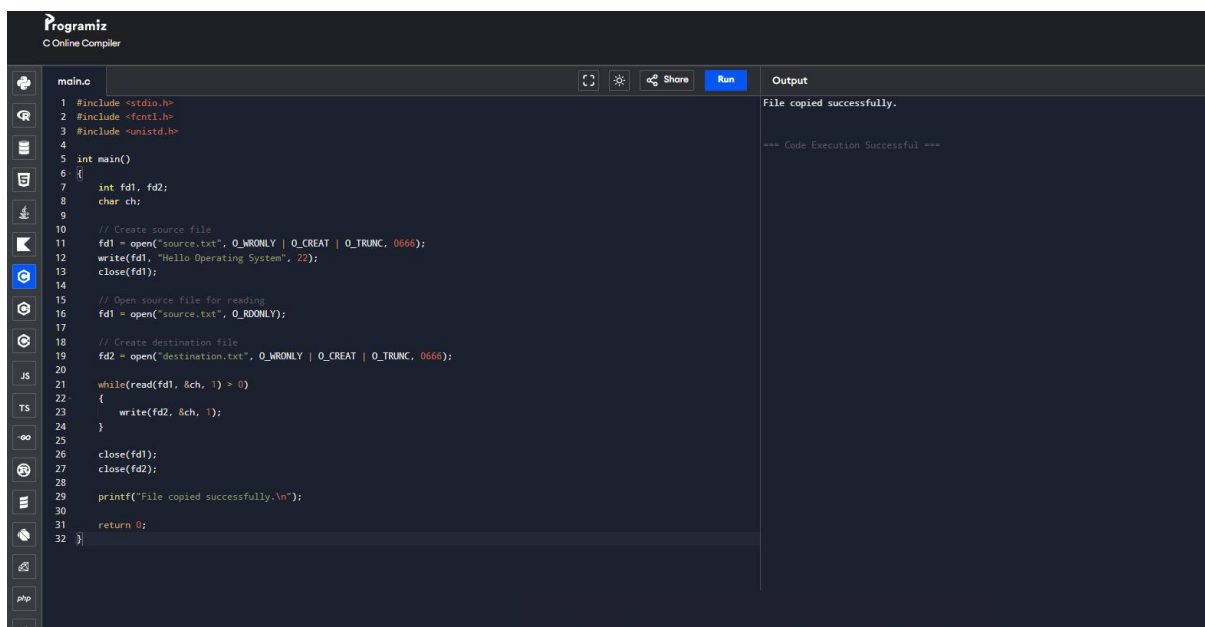
```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4
5 int main() {
6     pid_t pid;
7
8     // Create a new process
9     pid = fork();
10
11     if (pid < 0) {
12         // Fork failed
13         printf("Process creation failed!\n");
14     }
15     else if (pid == 0) {
16         // Child process
17         printf("\nChild Process\n");
18         printf("Child PID : %d\n", getpid());
19         printf("Parent PID : %d\n", getppid());
20     }
21     else {
22         // Parent process
23         printf("Parent Process\n");
24         printf("Parent PID : %d\n", getpid());
25         printf("Parent's Parent PID : %d\n", getppid());
26         printf("Child PID : %d\n", pid);
27     }
28
29     return 0;
30 }
```

Output

```
Parent Process
Child Process
Parent PID : 7415
Parent's Parent PID : 7404
Child PID : 7416
Child PID : 7416
Parent PID : 7415

=== Code Execution Successful ===
```

2. Identify the system calls to copy the content of one file to another and illustrate the same using a C program.



```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>
4
5 int main()
6 {
7     int fd1, fd2;
8     char ch;
9
10    // Create source file
11    fd1 = open("source.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);
12    write(fd1, "Hello Operating System", 22);
13    close(fd1);
14
15    // Open source file for reading
16    fd1 = open("source.txt", O_RDONLY);
17
18    // Create destination file
19    fd2 = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);
20
21    while(read(fd1, &ch, 1) > 0)
22    {
23        write(fd2, &ch, 1);
24    }
25
26    close(fd1);
27    close(fd2);
28
29    printf("File copied successfully.\n");
30
31    return 0;
32 }
```

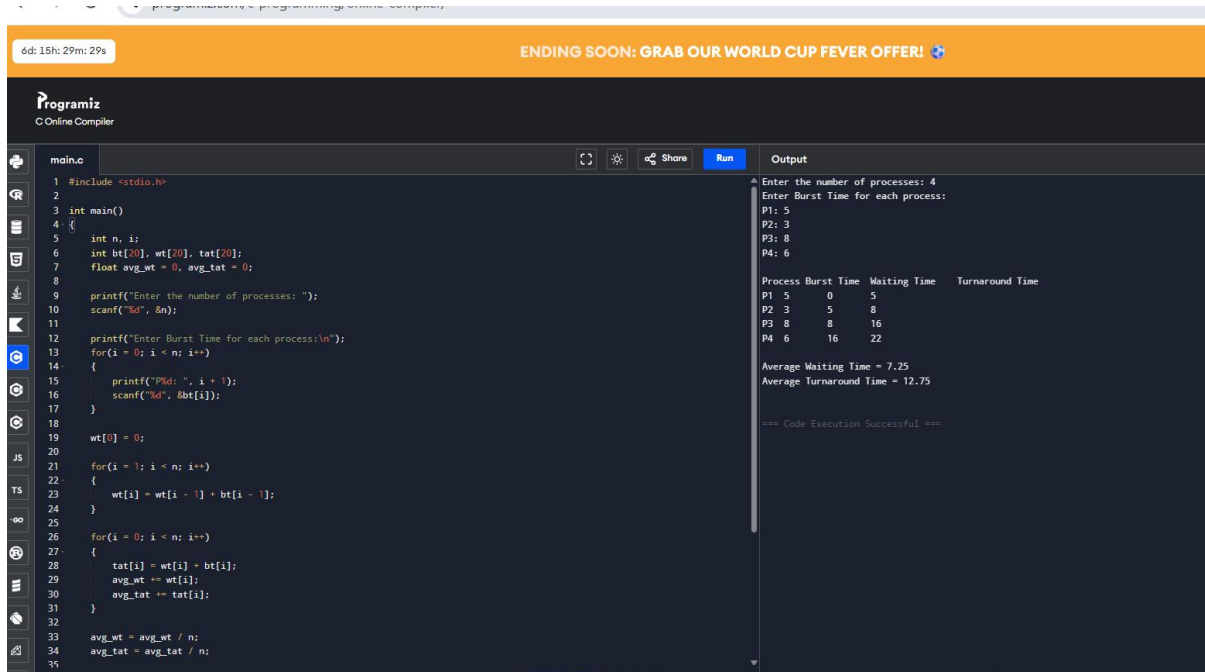
Output

```
File copied successfully.

=== Code Execution Successful ===
```

### 3. Design a CPU scheduling program with C using First Come First Served technique with the following considerations.

- a. All processes are activated at time 0.
- b. Assume that no process waits on I/O device



```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i;
6     int bt[20], wt[20], tat[20];
7     float avg_wt = 0, avg_tat = 0;
8
9     printf("Enter the number of processes: ");
10    scanf("%d", &n);
11
12    printf("Enter Burst Time for each process:\n");
13    for(i = 0; i < n; i++)
14    {
15        printf("P%d: ", i + 1);
16        scanf("%d", &bt[i]);
17    }
18
19    wt[0] = 0;
20
21    for(i = 1; i < n; i++)
22    {
23        wt[i] = wt[i - 1] + bt[i - 1];
24    }
25
26    for(i = 0; i < n; i++)
27    {
28        tat[i] = wt[i] + bt[i];
29        avg_wt += wt[i];
30        avg_tat += tat[i];
31    }
32
33    avg_wt = avg_wt / n;
34    avg_tat = avg_tat / n;
35}
```

Output

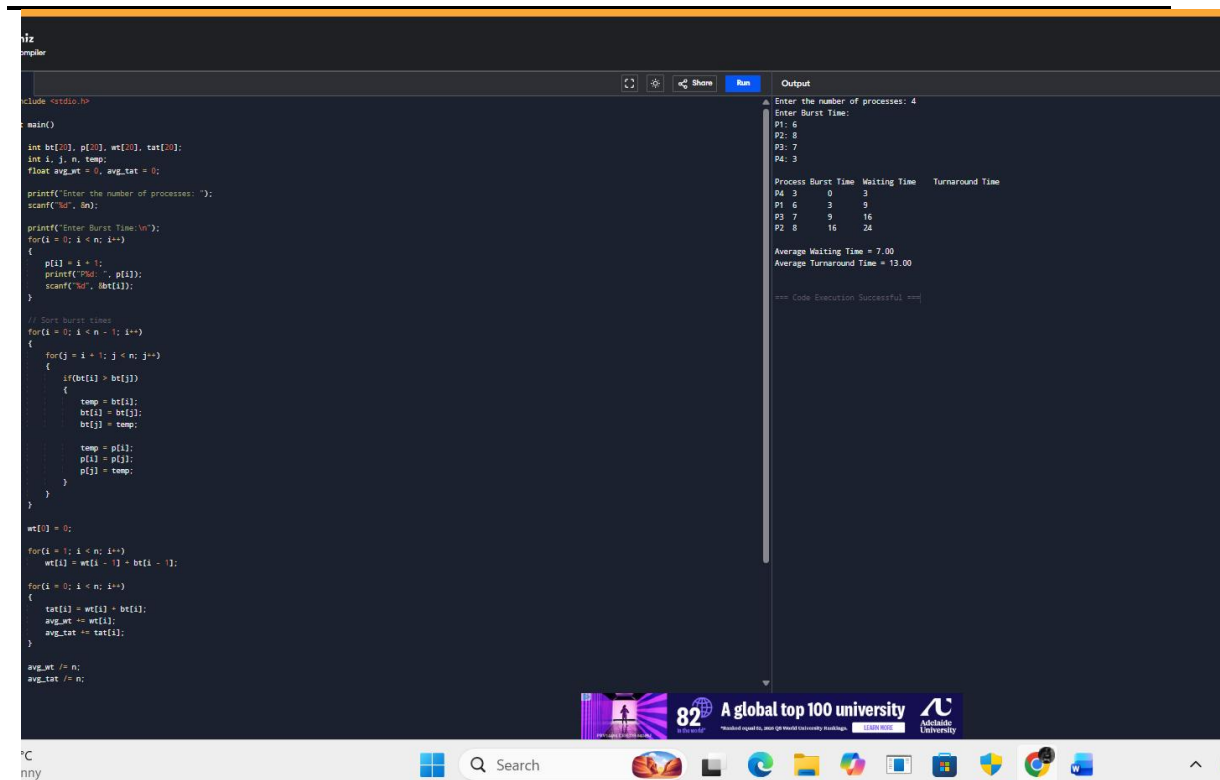
Enter the number of processes: 4  
Enter Burst Time for each process:  
P1: 5  
P2: 3  
P3: 8  
P4: 6

| Process | Burst Time | Waiting Time | Turnaround Time |
|---------|------------|--------------|-----------------|
| P1      | 5          | 0            | 5               |
| P2      | 3          | 5            | 8               |
| P3      | 8          | 8            | 16              |
| P4      | 6          | 16           | 22              |

Average Waiting Time = 7.25  
Average Turnaround Time = 12.75

=== Code Execution Successful ===

### 4. Construct a scheduling program with C that selects the waiting process with the smallest execution time to execute next



```
1 #include <stdio.h>
2
3 int main()
4 {
5     int bt[20], p[20], wt[20], tat[20];
6     int i, j, n, temp;
7     float avg_wt = 0, avg_tat = 0;
8
9     printf("Enter the number of processes: ");
10    scanf("%d", &n);
11
12    printf("Enter Burst Time:\n");
13    for(i = 0; i < n; i++)
14    {
15        p[i] = i + 1;
16        printf("P%d: ", p[i]);
17        scanf("%d", &bt[i]);
18    }
19
20    // Sort burst times
21    for(i = 0; i < n - 1; i++)
22    {
23        for(j = i + 1; j < n; j++)
24        {
25            if(bt[i] > bt[j])
26            {
27                temp = bt[i];
28                bt[i] = bt[j];
29                bt[j] = temp;
30
31                temp = p[i];
32                p[i] = p[j];
33                p[j] = temp;
34            }
35        }
36    }
37
38    wt[0] = 0;
39
40    for(i = 1; i < n; i++)
41    {
42        wt[i] = wt[i - 1] + bt[i - 1];
43    }
44
45    for(i = 0; i < n; i++)
46    {
47        tat[i] = wt[i] + bt[i];
48        avg_wt += wt[i];
49        avg_tat += tat[i];
50    }
51
52    avg_wt /= n;
53    avg_tat /= n;
54}
```

Output

Enter the number of processes: 4  
Enter Burst Time:  
P1: 6  
P2: 8  
P3: 7  
P4: 3

| Process | Burst Time | Waiting Time | Turnaround Time |
|---------|------------|--------------|-----------------|
| P4      | 3          | 0            | 3               |
| P1      | 6          | 3            | 9               |
| P3      | 7          | 9            | 16              |
| P2      | 8          | 16           | 24              |

Average Waiting Time = 7.00  
Average Turnaround Time = 13.00

=== Code Execution Successful ===

## 5. Construct a scheduling program with C that selects the waiting process with the highest priority to execute next.

← → ↺ programiz.com/c-programming/online-compiler/

dd:14h:36m:30s

ENDING SOON: GRAB OUR WORLD CUP FEVER OFFER! 🏆

Programiz  
C Online Compiler

```
main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     int bt[10], pr[10], wt[10], tat[10];
6     int i, j, n, temp;
7     float avg_wt = 0, avg_tat = 0;
8
9     printf("Enter the number of processes: ");
10    scanf("%d", &n);
11
12    printf("Enter Burst Time and Priority:\n");
13    for(i = 0; i < n; i++)
14    {
15        p[i] = i + 1;
16        printf("P%d Burst Time: ", p[i]);
17        scanf("%d", &bt[i]);
18        printf("P%d Priority: ", p[i]);
19        scanf("%d", &pr[i]);
20    }
21
22    // Sort by Priority (smaller number = higher priority)
23    for(i = 0; i < n; i++)
24    {
25        for(j = i + 1; j < n; j++)
26        {
27            if(pr[i] > pr[j])
28            {
29                temp = pr[i];
30                pr[i] = pr[j];
31                pr[j] = temp;
32
33                temp = bt[i];
34                bt[i] = bt[j];
35                bt[j] = temp;
36
37                temp = p[i];
38                p[i] = p[j];
39                p[j] = temp;
40            }
41        }
42    }
43
44    wt[0] = 0;
45    for(i = 1; i < n; i++)
46    {
47        wt[i] = wt[i - 1] + bt[i - 1];
48    }
49    for(i = 0; i < n; i++)
50    {
51        // ...
52    }
53}
```

Output

Enter the number of processes: 4  
Enter Burst Time and Priority:  
P1 Burst Time: 5  
P1 Priority: 2  
P2 Burst Time: 3  
P2 Priority: 1  
P3 Burst Time: 8  
P3 Priority: 4  
P4 Burst Time: 6  
P4 Priority: 3

| Process | Priority | Burst Time | Waiting Time | Turnaround Time |
|---------|----------|------------|--------------|-----------------|
| P2      | 1        | 3          | 0            | 3               |
| P1      | 2        | 5          | 3            | 8               |
| P4      | 3        | 6          | 8            | 14              |
| P3      | 4        | 8          | 14           | 22              |

Average Waiting Time = 6.25  
Average Turnaround Time = 11.75

==== Code Execution Successful ====

82 A global top 100 university

## 6. Construct a C program to implement preemptive priority scheduling algorithm

C Online Compiler

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int n, i, t = 0, c = 0;
5     int wt[10], bt[10], rt[10], pr[10];
6     int wt[10], tat[10], p, min;
7
8     printf("Enter number of processes: ");
9     scanf("%d", &n);
10
11    for(i = 0; i < n; i++) {
12        printf("P%d (AT BT Priority): ", i + 1);
13        scanf("%d%d", &rt[i], &bt[i], &pr[i]);
14        rt[i] = bt[i];
15    }
16
17    while(c < n) {
18        p = -1;
19        min = 999;
20
21        for(i = 0; i < n; i++) {
22            if((rt[i] <= t && rt[i] > 0 && pr[i] < min) {
23                min = pr[i];
24                p = i;
25            }
26        }
27
28        if(p == -1)
29            t++;
30        else {
31            rt[p]--;
32            t++;
33
34            if(rt[p] == 0) {
35                c++;
36                tat[p] = t - wt[p];
37                wt[p] = tat[p] + bt[p];
38            }
39        }
40    }
41
42    printf("\n\n");
43    for(i = 0; i < n; i++)
44        printf("P%d\t%d\t%d\n", i + 1, wt[i], tat[i]);
45
46    return 0;
47 }
```

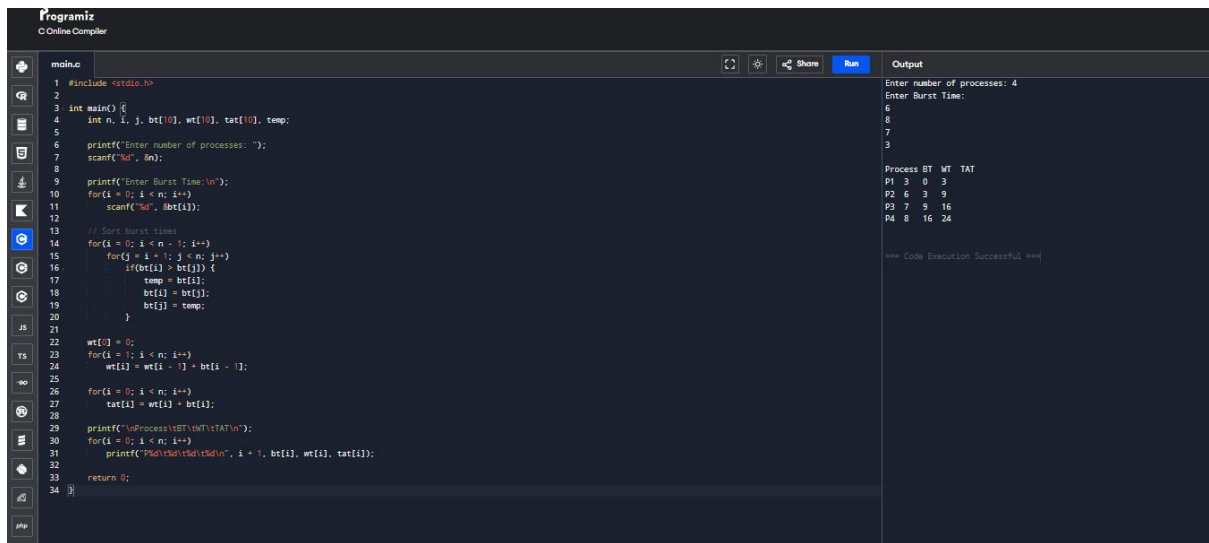
Output

Enter number of processes: 3  
P1 (AT BT Priority): 0 5 2  
P2 (AT BT Priority): 1 3 1  
P3 (AT BT Priority): 2 4 3

| P  | WT | TAT |
|----|----|-----|
| P1 | 3  | 8   |
| P2 | 0  | 3   |
| P3 | 6  | 10  |

==== Code Execution Successful ====

## 7. Construct a C program to implement a non-preemptive SJF algorithm.



The screenshot shows a C program implementing a non-preemptive Shortest Job First (SJF) scheduling algorithm. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j, bt[10], wt[10], tat[10], temp;
5
6     printf("Enter number of processes: ");
7     scanf("%d", &n);
8
9     printf("Enter Burst Time:\n");
10    for(i = 0; i < n; i++)
11        scanf("%d", &bt[i]);
12
13    // Sort burst times
14    for(i = 0; i < n - 1; i++)
15        for(j = i + 1; j < n; j++)
16            if(bt[i] > bt[j]) {
17                temp = bt[i];
18                bt[i] = bt[j];
19                bt[j] = temp;
20            }
21
22    wt[0] = 0;
23    for(i = 1; i < n; i++)
24        wt[i] = wt[i - 1] + bt[i - 1];
25
26    for(i = 0; i < n; i++)
27        tat[i] = wt[i] + bt[i];
28
29    printf("\nProcess\tBT\tWT\tTAT\n");
30    for(i = 0; i < n; i++)
31        printf("%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
32
33    return 0;
34 }
```

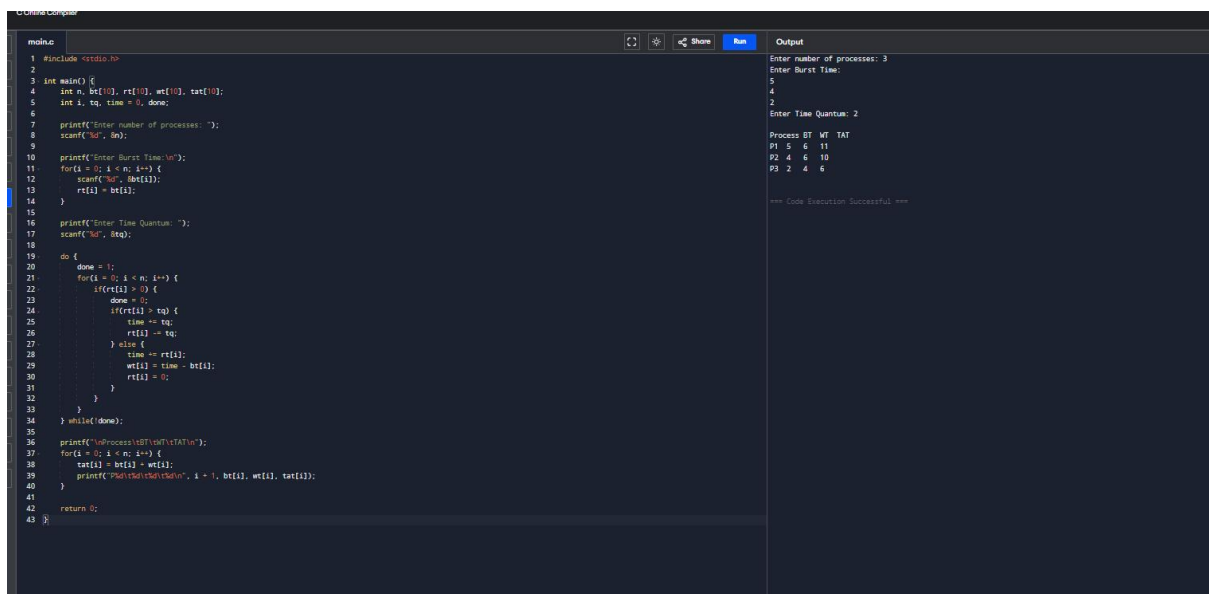
The output of the program is:

```
Enter number of processes: 4
Enter Burst Time:
6
8
7
3

Process BT WT TAT
P1 3 0 3
P2 6 3 9
P3 7 9 16
P4 8 16 24

*** Code Execution Successful ***
```

## 8. Construct a C program to simulate Round Robin scheduling algorithm with C.



The screenshot shows a C program implementing a Round Robin scheduling algorithm. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     int n, bt[10], rt[10], wt[10], tat[10];
5     int i, tq, time = 0, done;
6
7     printf("Enter number of processes: ");
8     scanf("%d", &n);
9
10    printf("Enter Burst Time:\n");
11    for(i = 0; i < n; i++) {
12        scanf("%d", &bt[i]);
13        rt[i] = bt[i];
14    }
15
16    printf("Enter Time Quantum: ");
17    scanf("%d", &tq);
18
19    do {
20        done = 1;
21        for(i = 0; i < n; i++) {
22            if(rt[i] > 0) {
23                done = 0;
24                if(rt[i] > tq) {
25                    time += tq;
26                    rt[i] -= tq;
27                } else {
28                    time += rt[i];
29                    wt[i] = time - bt[i];
30                    rt[i] = 0;
31                }
32            }
33        }
34    } while(!done);
35
36    printf("\nProcess\tBT\tWT\tTAT\n");
37    for(i = 0; i < n; i++) {
38        tat[i] = bt[i] + wt[i];
39        printf("%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
40    }
41
42    return 0;
43 }
```

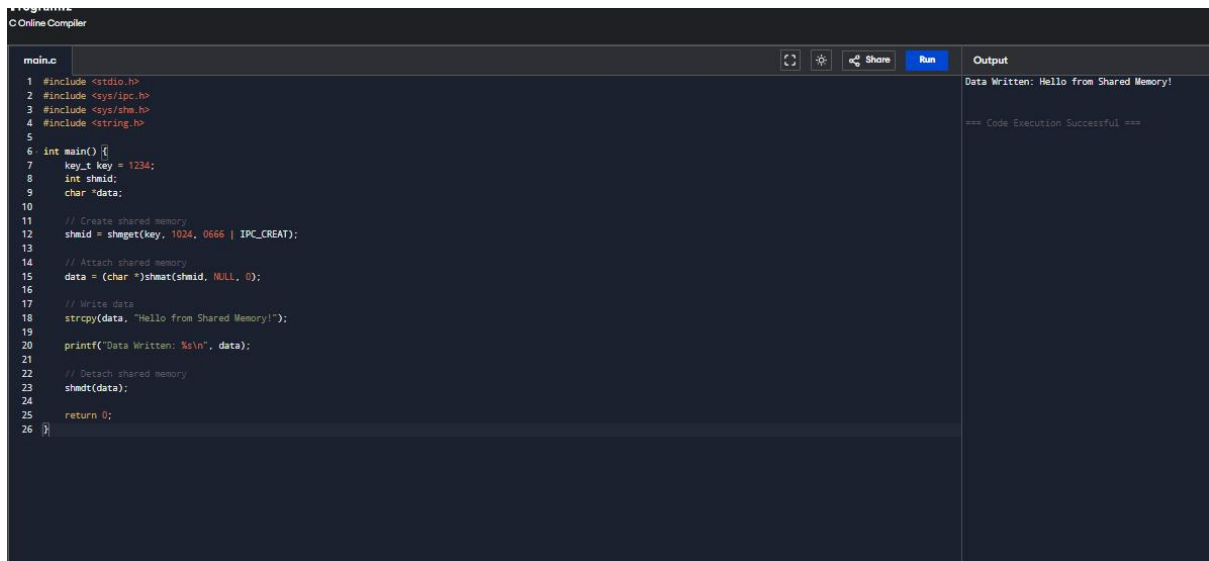
The output of the program is:

```
Enter number of processes: 3
Enter Burst Time:
5
4
2
Enter Time Quantum: 2

Process BT WT TAT
P1 5 6 11
P2 4 6 10
P3 2 4 6

*** Code Execution Successful ***
```

## 9. Illustrate the concept of inter-process communication using shared memory with a C program.



The screenshot shows a C program in a code editor. The program uses shared memory to store data. It includes headers for stdio, sys/ipc.h, sys/shm.h, and string. In the main function, it creates a shared memory segment with a key of 1234 and a size of 0666 bytes. It then attaches to this shared memory, writes the string "Hello from Shared Memory!" to it, and prints the data. Finally, it detaches from the shared memory and returns 0. The output window shows the printed message and a success message.

```
main.c
C Online Compiler

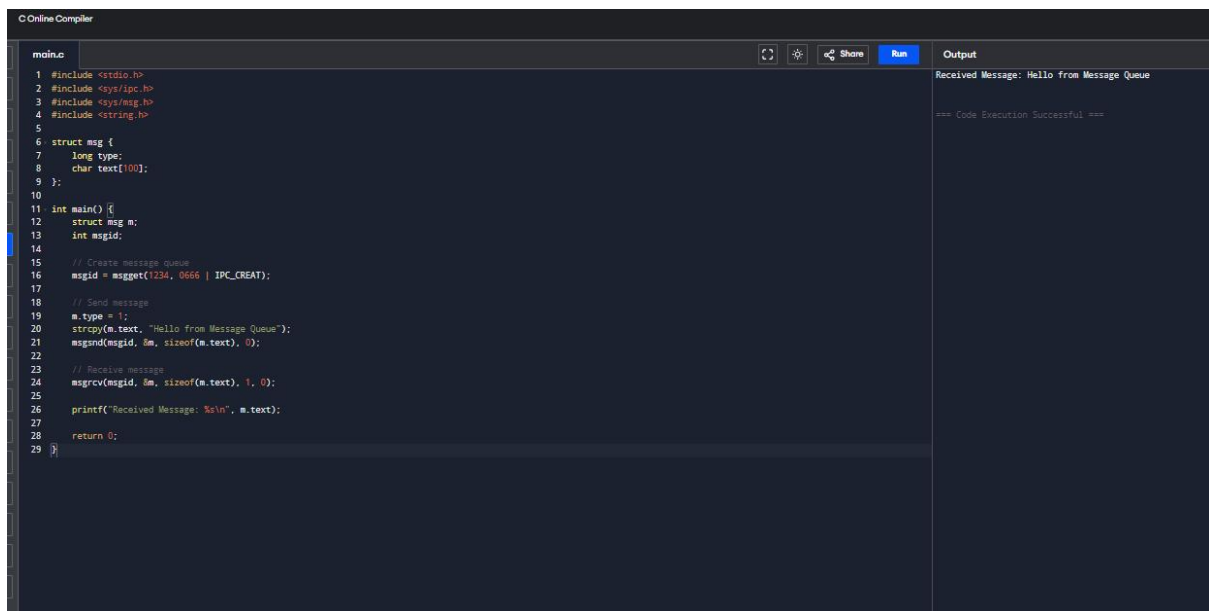
1 #include <stdio.h>
2 #include <sys/ipc.h>
3 #include <sys/shm.h>
4 #include <string.h>
5
6 int main() {
7     key_t key = 1234;
8     int shmid;
9     char *data;
10
11     // Create shared memory
12     shmid = shmget(key, 1024, 0666 | IPC_CREAT);
13
14     // Attach shared memory
15     data = (char *)shmat(shmid, NULL, 0);
16
17     // Write data
18     strcpy(data, "Hello from Shared Memory!");
19
20     printf("Data Written: %s\n", data);
21
22     // Detach shared memory
23     shmdt(data);
24
25     return 0;
26 }
```

Output

Data Written: Hello from Shared Memory!

=== Code Execution Successful ===

## 10. Illustrate the concept of inter-process communication using message queue with a C program.



The screenshot shows a C program in a code editor. The program uses message queues for inter-process communication. It includes headers for stdio, ipc.h, sys/ipc.h, sys/msg.h, and string. In the main function, it creates a message queue with a key of 1234 and a size of 0666 bytes. It then sends a message to the queue with the text "Hello from Message Queue". Finally, it receives the message and prints it. The output window shows the received message and a success message.

```
main.c
C Online Compiler

1 #include <stdio.h>
2 #include <sys/ipc.h>
3 #include <sys/msg.h>
4 #include <string.h>
5
6 struct msg {
7     long type;
8     char text[100];
9 };
10
11 int main() {
12     struct msg m;
13     int msgid;
14
15     // Create message queue
16     msgid = msgget(1234, 0666 | IPC_CREAT);
17
18     // Send message
19     m.type = 1;
20     strcpy(m.text, "Hello from Message Queue");
21     msgsnd(msgid, &m, sizeof(m.text), 0);
22
23     // Receive message
24     msgrcv(msgid, &m, sizeof(m.text), 1, 0);
25
26     printf("Received Message: %s\n", m.text);
27
28     return 0;
29 }
```

Output

Received Message: Hello from Message Queue

=== Code Execution Successful ===